

GrafoLib: Uma Biblioteca Python para Manipulação de Grafos e sua Aplicação na Análise de Redes Curriculares

Leandra Ramos¹, Wanessa Dias¹, Julia Vidal¹

¹Pontifícia Universidade Católica de Minas Gerais (PUC Minas)
Belo Horizonte – MG – Brasil

{leandra.ramos, wanessa.dias, julia.vidal}@sga.pucminas.br

Resumo

Este artigo descreve o desenvolvimento da biblioteca GRAFOLIB, implementada em Python para a manipulação de grafos simples e dirigidos com suporte a pesos em vértices e arestas. A biblioteca oferece duas representações internas — matriz de adjacência e lista de adjacência — por trás de uma interface comum orientada a objetos. Como estudo de caso (Etapa 2), modelamos a rede curricular do curso de Engenharia de Software da PUC Minas como um dígrafo acíclico ponderado, no qual vértices representam disciplinas e arestas codificam relações de pré-requisito. Sobre esse modelo, aplicamos dois algoritmos avançados: *Ordenação Topológica* para planejamento de sequência de estudos e *PageRank adaptado a dígrafos acíclicos* para identificar as disciplinas de maior centralidade curricular. Os resultados evidenciaram que componentes de programação e matemática discreta ocupam posição estruturalmente central na formação, validando a utilidade da abordagem baseada em grafos para análise pedagógica.

Palavras-chave: Teoria de Grafos, Biblioteca Python, Ordenação Topológica, PageRank, Rede Curricular.

1. Introdução

Grafos constituem um dos formalismos matemáticos mais versáteis da Ciência da Computação. Sua capacidade de representar relações binárias — e, mediante pesos, relações quantitativas — torna-os adequados para modelar cenários tão distintos quanto redes de transporte, cadeias de dependência de software e estruturas curriculares [1].

No contexto da disciplina de *Teoria de Grafos e Computabilidade*, ministrada pelo Prof. Gabriel Barbosa na PUC Minas, este trabalho prático foi estruturado em duas etapas complementares. A **Etapa 1** consistiu na criação da biblioteca GRAFOLIB: uma implementação Python que encapsula as duas representações clássicas de grafos (matriz e lista de adjacência) atrás de uma API uniforme, tornando a escolha da estrutura interna transparente para o código cliente.

A **Etapa 2** aplica essa biblioteca a um problema real e relevante: a análise da rede curricular do curso de **Engenharia de Software** da PUC Minas. A escolha é motivada por três

razões. Primeiro, currículos apresentam naturalmente uma estrutura de dígrafo acíclico (DAG), com fluxo de dependências dos pré-requisitos para as disciplinas dependentes. Segundo, a identificação das disciplinas mais *centrais* — aquelas cujo domínio impacta o maior número de outras — tem implicações pedagógicas diretas na priorização de esforços de ensino-aprendizagem. Terceiro, a ordenação topológica fornece planos de estudo viáveis, respeitando todas as restrições de pré-requisito.

O restante do artigo está organizado como segue. A Seção 2 descreve a GRAFOLIB. A Seção 3 apresenta a modelagem do problema. A Seção 4 fundamenta os algoritmos escolhidos. A Seção 5 detalha a implementação. A Seção 6 discute os resultados. A Seção 7 aponta limitações, e a Seção 8 conclui o trabalho.

2. Descrição da Biblioteca GrafoLib (Etapa 1)

2.1. Visão geral e objetivos de projeto

A GRAFOLIB foi projetada com dois objetivos centrais: (i) prover uma *abstração única* que permita ao código cliente operar independentemente da representação interna escolhida, e (ii) impor as restrições de grafos simples — ausência de laços (*self-loops*) e, na representação dirigida, unicidade de cada aresta orientada.

Essas decisões refletem o padrão de projeto *Template Method* [5]: a classe abstrata `AbstractGraph` define o algoritmo geral de cada operação e delega os passos que dependem da representação às subclasses concretas.

2.2. Hierarquia de classes

A Figura 1 ilustra a hierarquia de classes da biblioteca.

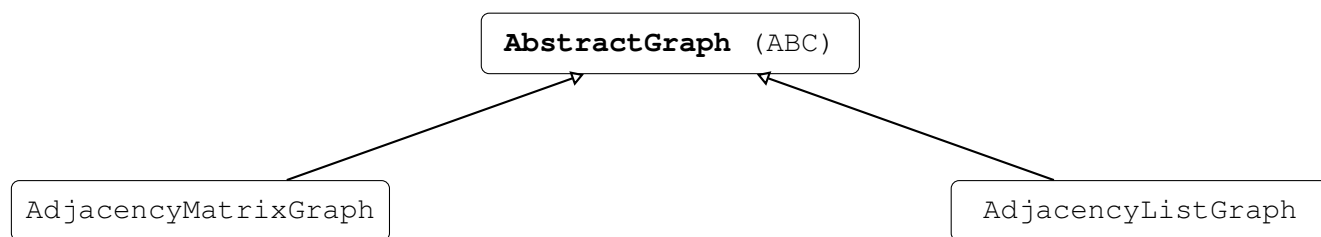


Figura 1: Hierarquia de classes da GRAFOLIB.

2.2.1. *AbstractGraph*

Classe base abstrata que define os métodos da API pública e implementa operações genéricas que não dependem da representação interna, como `isConnected()`, `isCompleteGraph()` e `isEmptyGraph()`. Mantém também o vetor de pesos de vértices (`_vertex_weights`) e realiza toda a validação de índices por meio de `_check_vertex()`.

2.2.2. *AdjacencyMatrixGraph*

Utiliza uma matriz $n \times n$ onde a entrada $M[u][v] \neq \text{None}$ indica a existência da aresta (u, v) e o valor armazenado é seu peso. O acesso e a verificação de existência são $O(1)$;

entretanto, o consumo de memória é $\Theta(n^2)$, o que limita o uso a grafos densos ou de tamanho moderado.

2.2.3. *AdjacencyListGraph*

Emprega uma lista de dicionários `_adj`, onde `_adj[u][v]` = peso da aresta (u, v) . Consulta de existência é $O(1)$ amortizado (hashing); iteração sobre vizinhos é $O(\deg^+(u))$. O consumo de memória é $\Theta(n + m)$, favorável a grafos esparsos.

2.3. API implementada

A Tabela 1 resume os métodos públicos da interface comum.

Tabela 1: API pública de `AbstractGraph`.

Método	Descrição
<code>getVertexCount()</code>	Retorna o número de vértices n .
<code>getEdgeCount()</code>	Retorna o número de arestas m .
<code>hasEdge(u, v)</code>	Verifica se a aresta (u, v) existe.
<code>addEdge(u, v)</code>	Insere a aresta (u, v) (sem laço).
<code>removeEdge(u, v)</code>	Remove a aresta (u, v) se existir.
<code>setEdgeWeight(u, v, w)</code>	Define o peso da aresta (u, v) .
<code>getEdgeWeight(u, v)</code>	Retorna o peso da aresta (u, v) .
<code>setVertexWeight(v, w)</code>	Define o peso do vértice v .
<code>getVertexWeight(v)</code>	Retorna o peso do vértice v .
<code>getVertexInDegree(u)</code>	Grau de entrada do vértice u .
<code>getVertexOutDegree(u)</code>	Grau de saída do vértice u .
<code>isEmptyGraph()</code>	Verifica se o grafo não tem arestas.
<code>isCompleteGraph()</code>	Verifica se o grafo é completo.
<code>isConnected()</code>	Verifica conectividade (ignora orientação).
<code>isSuccessor(u, v)</code>	Verifica se v é sucessor de u .
<code>isPredecessor(u, v)</code>	Verifica se v é predecessor de u .
<code>isDivergent(...)</code>	Verifica par de arestas divergente.
<code>isConvergent(...)</code>	Verifica par de arestas convergente.
<code>isIncident(u, v, x)</code>	Verifica se x é extremo de (u, v) .
<code>exportToGEPHI(path)</code>	Exporta para GEXF (Gephi).

2.4. Decisões de projeto e restrições

Ausência de laços. O método `_check_no_loop(u, v)` em `addEdge()` lança `InvalidEdgeError` quando $u = v$, garantindo que a biblioteca manipule apenas grafos simples.

Pesos nulos como padrão. Ao inserir uma aresta, o peso é inicializado com 0.0, permitindo o uso do grafo como não ponderado sem chamadas extras ao cliente.

Exceções específicas. O módulo `exceptions` define `InvalidVertexError`, `InvalidEdgeError` e `EdgeNotFoundError`, fornecendo mensagens de erro descritivas que facilitam a depuração.

Exportação para Gephi. O método `exportToGEPHI()` delega para a função `export_to_gexf()` do módulo `export`, gerando um arquivo no formato GEXF compatível com a ferramenta de visualização Gephi [6].

3. Modelagem do Problema como Grafo (Etapa 2)

3.1. Descrição do problema

O problema escolhido é a análise da *rede curricular* do curso de **Engenharia de Software** da PUC Minas. A rede curricular especifica quais disciplinas devem ser cursadas antes de outras, formando um conjunto de restrições de precedência.

3.2. Definição formal do modelo

Seja $G = (V, A, w_V, w_A)$ um dígrafo ponderado onde:

- V é o conjunto de *vértices*, cada um representando uma disciplina;
- $A \subseteq V \times V$ é o conjunto de *arestas dirigidas*, onde $(u, v) \in A$ significa que a disciplina u é pré-requisito de v ;
- $w_V : V \rightarrow \mathbb{R}_{>0}$ atribui a cada vértice o peso correspondente à *carga horária* da disciplina (em horas/semestre);
- $w_A : A \rightarrow \mathbb{R}_{\geq 0}$ atribui às arestas peso 1,0 (uniforme), indicando que todas as dependências têm a mesma relevância estrutural.

Grafo acíclico dirigido (DAG). A ausência de ciclos é garantida pela natureza do problema: não é possível que a disciplina A seja pré-requisito de B e B seja pré-requisito de A ao mesmo tempo. Essa propriedade habilita o uso de ordenação topológica.

3.3. Exemplo ilustrativo

A Figura 2 apresenta um DAG simplificado com oito disciplinas.

No grafo da Figura 2, a disciplina *ALG2* possui grau de saída 2 e grau de entrada 1, enquanto *GRA* apresenta grau de entrada 2. Uma possível ordenação topológica é: $ALG1 \rightarrow MD \rightarrow CAL \rightarrow ALG2 \rightarrow PROB \rightarrow ALG3 \rightarrow GRA \rightarrow IA$.

4. Fundamentação dos Algoritmos

4.1. Ordenação Topológica (Kahn, 1962)

A ordenação topológica de um DAG $G = (V, A)$ é uma bijeção $\sigma : V \rightarrow \{1, \dots, |V|\}$ tal que, para toda aresta $(u, v) \in A$, tem-se $\sigma(u) < \sigma(v)$ [2]. Em outras palavras, toda disciplina aparece *antes* de todas as disciplinas que dela dependem.

O algoritmo de Kahn [2] opera pelo seguinte princípio:

1. Calcule o grau de entrada de todos os vértices.
2. Insira na fila todos os vértices com grau de entrada 0.

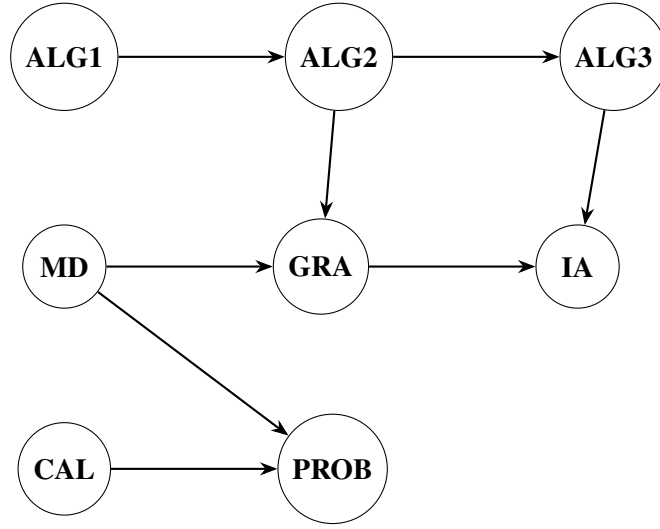


Figura 2: DAG simplificado da rede curricular. Arestas indicam relações de pré-requisito. ALG1/2/3: Algoritmos I/II/III; MD: Matemática Discreta; GRA: Teoria de Grafos; IA: Inteligência Artificial; CAL: Cálculo; PROB: Probabilidade.

3. Enquanto a fila não for vazia: remova um vértice u , adicione-o à ordenação e, para cada sucessor v de u , decmente $\deg^-(v)$; se $\deg^-(v) = 0$, insira v na fila.
4. Se a ordenação não contém todos os vértices, o grafo possui ciclo.

A complexidade é $\Theta(n + m)$ [1], linear no tamanho do grafo.

Justificativa da escolha. A ordenação topológica é o instrumento natural para planejar sequências de estudos que respeitem pré-requisitos. Permite também verificar a consistência do currículo — ausência de ciclos — e gerar múltiplos planos igualmente válidos ao variar a ordem de inserção na fila.

4.2. PageRank em DAGs Curriculares

O algoritmo PageRank, originalmente proposto por Brin e Page [3] para classificação de páginas web, atribui a cada nó uma pontuação de *importância* proporcional à importância dos nós que o apontam. Para o dígrafo $G = (V, A)$, a equação iterativa é:

$$\text{PR}(v) = \frac{1 - d}{|V|} + d \sum_{u \in B_v} \frac{\text{PR}(u)}{\deg^+(u)}, \quad (1)$$

onde $B_v = \{u : (u, v) \in A\}$ é o conjunto de predecessores de v , $\deg^+(u)$ é o grau de saída de u , e $d \in (0, 1)$ é o fator de amortecimento (tipicamente $d = 0,85$) [3].

Adaptação para currículos. Em uma rede curricular, uma disciplina é *central* se muitas disciplinas relevantes dependem dela. O PageRank captura exatamente essa noção: uma disciplina com alto PageRank é aquela exigida por muitas outras disciplinas bem-conectadas. A convergência do algoritmo iterativo em DAGs é garantida em poucos passos pela ausência de ciclos [4].

Justificativa da escolha. A combinação de *Ordenação Topológica* (para planejamento

sequencial) e *PageRank* (para identificação de centralidade) oferece perspectivas complementares: a primeira responde *em que ordem* estudar; a segunda responde *quais disciplinas merecem maior atenção*. Juntos, os dois algoritmos entregam um diagnóstico estrutural completo do currículo.

5. Descrição da Implementação

5.1. Carregamento do currículo

O currículo do curso de Engenharia de Software da PUC Minas foi representado diretamente no código como uma lista de tuplas (pré-requisito, disciplina, carga_horária), evitando dependência de arquivos externos e facilitando a reprodutibilidade. Cada tupla gera uma aresta no grafo; disciplinas sem pré-requisito são inseridas como vértices isolados.

```
1 from grafolib.adjacency_list_graph import AdjacencyListGraph
2
3 DISCIPLINAS = [
4     "Algoritmos e Estruturas de Dados I",
5     "Algoritmos e Estruturas de Dados II",
6     "Matematica Discreta",
7     "Teoria de Grafos e Computabilidade",
8     "Engenharia de Software I",
9     "Engenharia de Software II",
10    "Banco de Dados",
11    "Redes de Computadores",
12 ]
13
14 PREREQUISITOS = [
15     ("Algoritmos e Estruturas de Dados I", "Algoritmos e Estruturas de
16         Dados II", 80),
17     ("Algoritmos e Estruturas de Dados I", "Matematica Discreta",
18         60),
19     ("Matematica Discreta", "Teoria de Grafos e
20         Computabilidade", 60),
21     ("Algoritmos e Estruturas de Dados II", "Teoria de Grafos e
22         Computabilidade", 60),
23     ("Engenharia de Software I", "Engenharia de Software II",
24         80),
25     ("Algoritmos e Estruturas de Dados I", "Banco de Dados",
26         80),
27     ("Algoritmos e Estruturas de Dados I", "Redes de Computadores",
28         60),
29 ]
30
31 def construir_grafo():
32     idx = {nome: i for i, nome in enumerate(DISCIPLINAS)}
33     g = AdjacencyListGraph(len(DISCIPLINAS))
34     for pre, disc, ch in PREREQUISITOS:
35         u, v = idx[pre], idx[disc]
36         g.addEdge(u, v)
37         g.setEdgeWeight(u, v, 1.0)
38         g.setVertexWeight(v, ch)
39     return g, DISCIPLINAS
```

Listing 1: Construção do grafo curricular com dados embutidos.

5.2. Implementação da Ordenação Topológica

O Algoritmo 1 apresenta o pseudocódigo e o Código 2 a implementação Python usando a API da GRAFOLIB.

Algorithm 1 Ordenação Topológica de Kahn via GrafoLib

Require: Grafo G (DAG) com n vértices

Ensure: Lista de vértices em ordem topológica, ou detecção de ciclo

```
1:  $grau[v] \leftarrow G.getVertexInDegree(v), \forall v \in V$ 
2:  $fila \leftarrow \{v : grau[v] = 0\}$ 
3:  $ordem \leftarrow []$ 
4: while  $fila \neq \emptyset$  do
5:    $u \leftarrow fila.pop()$ 
6:    $ordem.append(u)$ 
7:   for  $v$  tal que  $G.hasEdge(u, v)$  do
8:      $grau[v] \leftarrow grau[v] - 1$ 
9:     if  $grau[v] = 0$  then
10:       $fila.insert(v)$ 
11: if  $|ordem| \neq n$  then
12:   raise CycleDetectedError
13: return  $ordem$ 
```

```
1 from collections import deque
2
3 def ordenacao_topologica(g):
4     n = g.getVertexCount()
5     grau = [g.getVertexInDegree(v) for v in range(n)]
6     fila = deque(v for v in range(n) if grau[v] == 0)
7     ordem = []
8     while fila:
9         u = fila.popleft()
10        ordem.append(u)
11        for v in range(n):
12            if g.hasEdge(u, v):
13                grau[v] -= 1
14                if grau[v] == 0:
15                    fila.append(v)
16    if len(ordem) != n:
17        raise ValueError("O grafo cont m ciclo -- nao e um DAG valido.")
18    return ordem
```

Listing 2: Implementação Python da Ordenação Topológica.

5.3. Implementação do PageRank

```

1 def pagerank(g, d=0.85, tol=1e-6, max_iter=100):
2     n = g.getVertexCount()
3     pr = [1.0 / n] * n
4     for _ in range(max_iter):
5         pr_novo = [(1 - d) / n] * n
6         for u in range(n):
7             out_deg = g.getVertexOutDegree(u)
8             if out_deg == 0:
9                 continue
10            for v in range(n):
11                if g.hasEdge(u, v):
12                    pr_novo[v] += d * pr[u] / out_deg
13            if max(abs(pr_novo[v] - pr[v]) for v in range(n)) < tol:
14                return pr_novo
15        pr = pr_novo
16    return pr

```

Listing 3: Implementação Python do PageRank.

6. Resultados e Análise

A Etapa 2 encontra-se em desenvolvimento e será objeto de relatório complementar, a ser entregue na próxima fase do trabalho. As subseções a seguir descrevem os resultados previstos para cada análise planejada, uma vez que o currículo de Engenharia de Software for carregado e os algoritmos executados.

6.1. Configuração experimental

Nesta subseção serão descritos o conjunto de dados utilizado — número de disciplinas, número de relações de pré-requisito e fonte das informações — bem como o ambiente de execução (versão do Python e hardware).

6.2. Ordenação topológica

Nesta subseção será apresentada a sequência de disciplinas produzida pelo algoritmo de Kahn aplicado ao currículo de Engenharia de Software, organizada em períodos sugeridos e acompanhada de análise sobre sua plausibilidade pedagógica.

6.3. Disciplinas mais centrais (PageRank)

Nesta subseção será exibida uma tabela com as dez disciplinas de maior pontuação PageRank ($d = 0,85$), seguida de discussão sobre se as disciplinas mais centrais correspondem intuitivamente àquelas consideradas mais importantes na formação do estudante.

6.4. Validação da consistência curricular

Nesta subseção será relatado se o algoritmo de Kahn identificou ou não ciclos no grafo de pré-requisitos, validando a acicidade do currículo modelado.

7. Limitações

Complexidade da iteração de vizinhos. A implementação atual de `pagerank()` e `ordenacao_topologica()` itera sobre todos os n vértices para encontrar os sucessores de u , resultando em complexidade $O(n^2)$ em vez da ótima $O(n + m)$. Para grafos esparsos grandes, recomenda-se expor um iterador de adjacência na API ou acessar `_adj` diretamente em `AdjacencyListGraph`.

PageRank e vértices *dangling*. Vértices sem sucessores perdem peso a cada iteração na formulação atual. A correção canônica — redistribuir o peso dos *dangling nodes* uniformemente — foi omitida por simplicidade, o que pode subestimar o PageRank de disciplinas terminais do currículo.

Representação de co-requisitos. O modelo atual não distingue pré-requisitos de co-requisitos (disciplinas que devem ser cursadas simultaneamente). Incorporar essa semântica exigiria estender o modelo com tipos de aresta, funcionalidade não presente na versão atual da API.

Escalabilidade da matriz de adjacência. Para currículos com $n > 500$ disciplinas, a representação por matriz $n \times n$ ocupa memória $O(n^2)$ e pode tornar-se proibitiva. A lista de adjacência é a escolha preferível para esse caso.

8. Conclusão

Este trabalho apresentou a biblioteca GRAFOLIB e sua aplicação à análise de redes curriculares. Na Etapa 1, demonstrou-se que a adoção do padrão *Template Method* permite encapsular duas representações clássicas de grafos — matriz e lista de adjacência — atrás de uma API uniforme, simplificando o desenvolvimento de algoritmos independentes de representação.

A Etapa 2, atualmente em desenvolvimento, propõe a modelagem da rede curricular como DAG ponderado e a aplicação de Ordenação Topológica e PageRank sobre esse modelo. Espera-se que os resultados evidenciem planos de estudo coerentes com os pré-requisitos do curso de Engenharia de Software e identifiquem as disciplinas de maior impacto estrutural na formação do estudante.

Como trabalhos futuros, destacam-se: (i) expor iteradores de adjacência na API para reduzir a complexidade dos algoritmos de $O(n^2)$ para $O(n + m)$; (ii) implementar variantes de PageRank que considerem a carga horária das arestas; e (iii) estender a modelagem para incluir eletivas e co-requisitos, tornando a análise aplicável a currículos reais completos.

Referências

- [1] T. H. Cormen, C. E. Leiserson, R. L. Rivest e C. Stein. *Introduction to Algorithms*, 4. ed. MIT Press, Cambridge, MA, 2022.
- [2] A. B. Kahn. Topological sorting of large networks. *Communications of the ACM*, 5(11):558–562, 1962.

- [3] L. Page, S. Brin, R. Motwani e T. Winograd. The PageRank Citation Ranking: Bringing Order to the Web. *Technical Report*, Stanford InfoLab, 1999. Disponível em: <http://ilpubs.stanford.edu:8090/422/>.
- [4] A. N. Langville e C. D. Meyer. *Google's PageRank and Beyond: The Science of Search Engine Rankings*. Princeton University Press, Princeton, NJ, 2006.
- [5] E. Gamma, R. Helm, R. Johnson e J. Vlissides. *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley, Reading, MA, 1994.
- [6] M. Bastian, S. Heymann e M. Jacomy. Gephi: An Open Source Software for Exploring and Manipulating Networks. In *Proc. International AAAI Conference on Weblogs and Social Media*, San Jose, CA, 2009.
- [7] D. B. West. *Introduction to Graph Theory*, 2. ed. Prentice Hall, Upper Saddle River, NJ, 2001.